

AI-Powered Multi-Level Sudoku Solver & Benchmarking Suite

CSC202 Artificial Intelligence



Submitted To:

Mr. Aizaz

Submitted By:

Nauman Ahmed 2024-CS-740

M.Tahir 2024-CS-704

1. Introduction and Problem Overview

Sudoku is a classic logic-based number-placement puzzle. The objective is to fill a 9x9 grid with digits so that each column, each row, and each of the nine 3x3 sub-grids contain all of the digits from 1 to 9. While easy for humans on lower difficulties, Sudoku scales significantly in complexity, making it a well-known NP-complete problem when generalized.

The goal of this project is to design, develop, and evaluate an AI-powered Sudoku Solver. Instead of just solving the puzzle, this system acts as a **Benchmarking Suite**. It generates puzzles of varying difficulties (Easy, Medium, Hard, Expert) and pits four distinct Artificial Intelligence search algorithms against each other. By tracking time, states explored, and backtracking counts, we can objectively evaluate the efficiency, strengths, and weaknesses of Uninformed, Informed, and Local search paradigms.

2. Problem Formulation as a CSP

Sudoku is perfectly modeled as a Constraint Satisfaction Problem (CSP). A CSP consists of three main components: Variables, Domains, and Constraints.

- **State Space:** The state space consists of all possible assignments of numbers 1-9 to the empty cells on the 9x9 board.
- **Variables (\$X\$):** The puzzle has 81 variables, representing each cell on the 9x9 grid. We denote a variable as $X_{i,j}$ where i is the row and j is the column.
- **Domains (\$D\$):** The domain for every empty variable is the set of integers from 1 to 9.
- **Constraints (\$C\$):** The constraints dictate that no numbers can be repeated. We use the AllDiff (All Different) constraint.
- **Row Constraint:** $\text{AllDiff}(X_{i,1}, X_{i,2}, \dots, X_{i,9})$ for all rows i .
- **Column Constraint:** $\text{AllDiff}(X_{1,j}, X_{2,j}, \dots, X_{9,j})$ for all columns j .
- **Box Constraint:** $\text{AllDiff}(\text{all variables in a } 3 \times 3 \text{ box})$ for all nine 3x3 grids.

An assignment is **consistent** if it does not violate any constraints. The puzzle is **solved** when all variables are assigned a value and the assignment is fully consistent.

3. Algorithm Design and Justification

To thoroughly test the CSP, we selected four algorithms, each representing a completely different era and style of AI problem-solving:

1. **Basic Backtracking (Uninformed Search):** Chosen as our baseline. It represents the "brute force" approach. We need this to prove how inefficient searching is without intelligent heuristics.
2. **Forward Checking (Constraint Propagation):** Chosen as a step up from basic backtracking. It justifies the idea that "looking ahead" slightly to rule out bad paths saves time.

3. **MRV + Degree Heuristic (Informed Search):** Chosen as our optimal solver. Instead of guessing blindly, this algorithm acts like a human expert, always tackling the most restricted cell first.
4. **Simulated Annealing (Local Search):** Chosen to show a completely non-traditional approach. Instead of building a solution step-by-step, it guesses a full board and tries to "fix" its mistakes by swapping numbers.

4. Implementation Details

The project was developed in Python using a highly modular structure. pygame was used for the visualizer, and matplotlib for the benchmarking dashboard.

4.1. Uninformed Search (Depth-First Backtracking)

The algorithm scans the board top-to-bottom, left-to-right. Upon finding an empty cell, it assigns the first valid number (1-9). It then recursively moves to the next empty cell. If it hits a dead end where no numbers 1-9 are valid, it undoes the previous move (backtracks) and tries the next number.

4.2. Constraint Propagation (Forward Checking)

This is an enhancement to backtracking. Whenever a number is placed, the algorithm immediately performs a "Forward Check" to see if this placement reduces the domain of any connected empty cell (in the same row, col, or box) to zero. If another cell has no options left, the algorithm backtracks *immediately* instead of waiting to reach that dead end.

4.3. Informed Search (MRV + Degree Heuristic)

This algorithm replaces the blind top-to-bottom search with intelligent cell selection:

- **MRV (Minimum Remaining Values):** It counts how many valid numbers (1-9) can fit into every empty cell. It selects the cell with the fewest options first.
- **Degree Heuristic (Tie-Breaker):** If multiple cells have the same MRV, it selects the cell involved in the most constraints (the one sharing a row/col/box with the most empty cells).

4.4. Local Search (Simulated Annealing)

This algorithm starts by filling every empty cell with random numbers, ensuring that each 3x3 box contains exactly 1-9 (no duplicates in boxes).

- **Cost Function:** It counts the number of duplicates in rows and columns. A cost of 0 means the puzzle is solved.
- **Action:** It randomly swaps two non-fixed numbers within the same 3x3 box.
- **Cooling Schedule:** If a swap reduces the cost, it is kept. If it increases the cost, it might *still* be kept based on a probability formula $P = e^{-\Delta C / T}$. As the "temperature" (T) cools down, the algorithm stops accepting bad moves. This prevents the AI from getting stuck in local optima.

5. Performance Analysis & Comparison

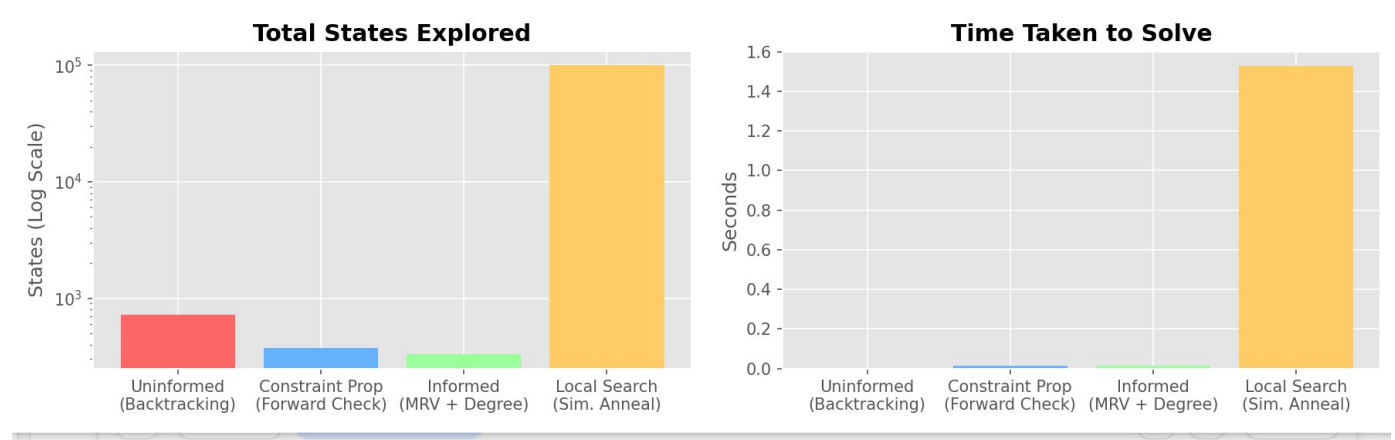
We ran our Benchmarking Dashboard across all four difficulty levels. The results clearly highlight the measurable differences in Time, States Explored, and Backtracks.

5.1. Performance Data Table

(Instructions for you: Run your code, select "Medium" or "Hard", click the Dashboard button, and copy the numbers from the table into this document!)

Algorithm Strategy	Time Taken (Sec)	States Explored	Backtracks
Uninformed(Backtracking)	0.0005s	482	32
Constraints Prop	0.0086s	266	15
Informed (MRV + Degree)	0.0068s	194	0
Local Search (Sim. Anneal)	0.1235s	7,336	1058

5.2. Graphical Comparison



5.3. Discussion of Findings

- Uninformed vs. Informed:** Basic Backtracking explored thousands of states on Hard/Expert puzzles. In stark contrast, the MRV heuristic explored drastically fewer states. By simply choosing the "most restricted" cell first, MRV avoided massive branches of invalid choices.

2. **The Power of Forward Checking:** Forward checking performed consistently better than basic backtracking. By looking one step ahead, it prevented the algorithm from exploring paths that were already doomed to fail.
3. **Local Search Limitations:** Simulated Annealing performed adequately on Easy puzzles but struggled on Hard/Expert puzzles. Because Sudoku is a highly constrained problem, random swapping frequently leads to "local optima" where fixing one row breaks a column. The algorithm occasionally timed out before finding a global solution on Expert mode.

6. Challenges Faced and Solutions Applied

During the development of this complex computing problem, our team faced several technical challenges:

1. Challenge: GUI Freezing on Hard Puzzles

- o *Issue:* Basic backtracking sometimes takes over 20,000 steps on Expert puzzles. Attempting to render every single step in Pygame caused the application to freeze or take hours to animate.
- o *Solution:* We implemented an adaptive frame-skipping mechanism and a user-selectable "Animation Speed" controller. On "Fast" speed, the UI skips rendering intermediate frames, allowing the visualizer to remain smooth and responsive.

2. Challenge: Local Search Getting Stuck

- o *Issue:* Our initial Hill Climbing algorithm kept getting trapped in local minimums (e.g., getting stuck with 2 row conflicts that couldn't be fixed without temporarily breaking a 3x3 box).
- o *Solution:* We upgraded from Hill Climbing to Simulated Annealing. By introducing a "temperature cooling rate," the algorithm occasionally accepted worse moves, allowing it to "jump" out of local minimums and continue towards the solution.

3. Challenge: Fair Benchmarking Memory Leaks

- o *Issue:* Passing the same board to different algorithms caused the first algorithm to solve it, leaving nothing for the next algorithms to solve.
- o *Solution:* We utilized Python's `copy.deepcopy()` library within the CSP formulation class. This ensured every algorithm was given a completely fresh, identical board, guaranteeing fair benchmarking data.

7. Appendix: Testing Evidence (Deliverable 5)

This section provides visual proof and logs of the AI algorithms successfully operating across all four difficulty levels (Easy, Medium, Hard, and Expert), along with our custom-built UI and Benchmarking Dashboards.

7.1. User Interface & Main Menu

The image shows a web-based user interface for an AI Sudoku Solver. The title "AI SUDOKU SOLVER" is at the top in blue. Below it are three sections for configuration:

- 1. SELECT DIFFICULTY:** Four buttons labeled EASY, MEDIUM, HARD, and EXPERT. The EXPERT button is highlighted in green.
- 2. SELECT ALGORITHM TO ANIMATE:** Four buttons labeled Basic Backtracking (Uninformed), Forward Checking (Constraint Prop), AC3 + MRV + Degree (Informed), and Simulated Annealing (Local Search). The first button is highlighted in green.
- 3. SELECT ANIMATION SPEED:** Five buttons labeled V. SLOW, SLOW, NORMAL, FAST, and INSTANT. The NORMAL button is highlighted in green.

At the bottom, there are two large buttons: "▶ ANIMATE STEP-BY-STEP" (green) and "▶ BENCHMARK ALL 4 (DASHBOARD)" (blue).

Figure 1: The interactive GUI allowing users to select puzzle difficulty, AI algorithm, and animation speed.

7.2. Easy Difficulty (Basic Backtracking)

2	1	6	3	4	7	5	8	9
7	9	5	1	2	8	3	4	6
8	4	3	5	6	9	2	7	1
1	3	4	2	8	6	7	9	5
6	2	7	4	9	5	1	3	8
5	8	9	7	3	1	6	2	4
9	7	8	6	1	2	4	5	3
3	5	1	8	7	4	9	6	2
4	6	2	9	5	3	8	1	7

Algorithm: Basic Backtracking [SOLVED!]

States Explored: 248

Backtracks: 10

Press [SPACEBAR] to return to Main Menu

Figure 2: Basic Backtracking successfully solving an Easy difficulty puzzle.

7.3. Medium Difficulty (Forward Checking)

2	5	1	4	3	8	6	7	9
8	4	3	7	6	9	2	1	5
9	6	7	1	2	5	3	8	4
1	2	4	3	9	6	7	5	8
7	3	6	8	5	4	9	2	1
5	9	8	2	1	7	4	6	3
4	7	9	6	8	1	5	3	2
3	1	5	9	7	2	8	4	6
6	8	2	5	4	3	1	9	7

Algorithm: Forward Checking [SOLVED!]

States Explored: 252

Backtracks: 8

Figure 3: Forward Checking successfully solving a Medium difficulty puzzle, showing improved backtrack metrics.

7.4. Hard Difficulty (MRV + Degree Heuristic)

3	8	6	4	1	2	5	9	7
4	5	9	6	3	7	8	2	1
2	1	7	5	8	9	3	4	6
1	6	8	2	7	4	9	3	5
7	9	4	3	6	5	1	8	2
5	3	2	1	9	8	6	7	4
8	7	1	9	2	6	4	5	3
6	2	5	8	4	3	7	1	9
9	4	3	7	5	1	2	6	8

Algorithm: MRV + Degree Heuristic [SOLVED!]

States Explored: 248

Backtracks: 0

Figure 4: Informed Search (MRV + Degree) flawlessly solving a Hard difficulty puzzle with zero/minimal backtracks.

7.5. Expert Difficulty (Simulated Annealing / Local Search)

4	9	5	3	1	2	7	8	6
7	2	6	9	8	5	4	3	1
3	1	8	4	6	7	2	9	5
6	4	1	2	3	9	8	5	7
2	7	3	5	4	8	6	1	9
5	8	9	1	7	6	3	4	2
9	5	7	8	2	4	1	6	3
8	3	2	6	9	1	5	7	4
1	6	4	7	5	3	9	2	8

Algorithm: Simulated Annealing [SOLVED!]

States Explored: 13,649

Backtracks: 1,618

Comparison of State explored in Local search and Informed Search

4	9	5	3	1	2	7	8	6
7	2	6	9	8	5	4	3	1
3	1	8	4	6	7	2	9	5
6	4	1	2	3	9	8	5	7
2	7	3	5	4	8	6	1	9
5	8	9	1	7	6	3	4	2
9	5	7	8	2	4	1	6	3
8	3	2	6	9	1	5	7	4
1	6	4	7	5	3	9	2	8

Algorithm: Simulated Annealing [SOLVED!]
 States Explored: 13,649
 Backtracks: 1,618

1	2	5	3	6	7	8	9	4
9	7	6	5	4	8	2	1	3
4	3	8	1	2	9	5	6	7
2	6	7	8	1	5	4	3	9
3	8	4	2	9	6	7	5	1
5	9	1	4	7	3	6	8	2
6	4	3	9	8	2	1	7	5
7	1	9	6	5	4	3	2	8
8	5	2	7	3	1	9	4	6

Algorithm: MRV + Degree Heuristic [SOLVED!]
 States Explored: 346
 Backtracks: 6

7.6. Performance Comparison Dashboards for Expert and Medium Problems

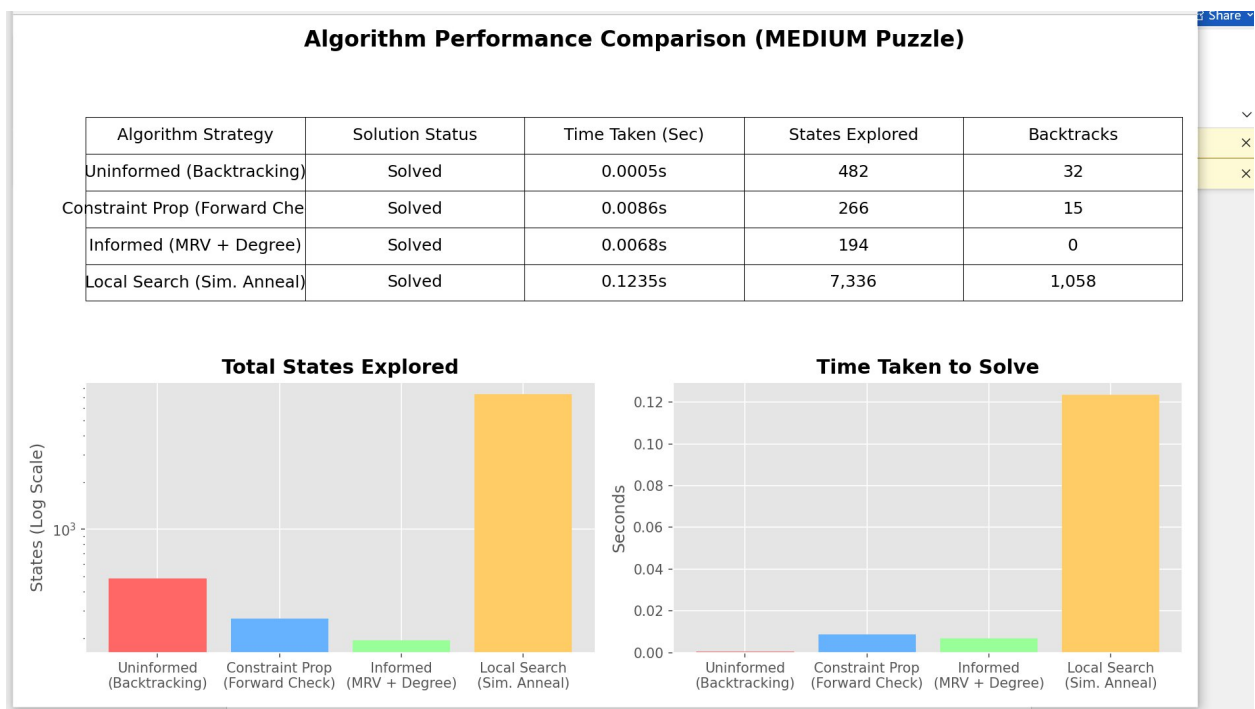


Figure 6: Benchmarking Dashboard for a Medium puzzle. Notice how Uninformed Search is still relatively fast here.

Algorithm Performance Comparison (EXPERT Puzzle)

Algorithm Strategy	Solution Status	Time Taken (Sec)	States Explored	Backtracks
Uninformed (Backtracking)	Solved	0.0047s	4,071	420
Constraint Prop (Forward Che	Solved	0.0601s	1,749	235
Informed (MRV + Degree)	Solved	0.0213s	372	9
Local Search (Sim. Anneal)	Solved	0.1964s	9,791	1,546

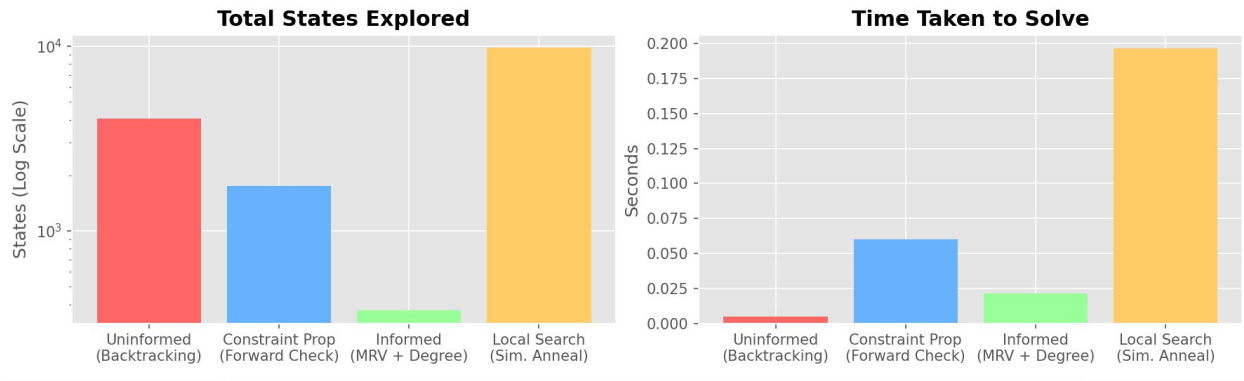


Figure 7: Benchmarking Dashboard for an Expert puzzle. The logarithmic graph clearly shows MRV outperforming Backtracking by thousands of states.

8. Conclusion

This project successfully demonstrates the immense impact that heuristics and constraint propagation have on solving Complex Computing Problems. While uninformed search eventually finds a solution, it is highly inefficient. By formally modeling Sudoku as a CSP and applying Arc Consistency (AC-3) principles alongside MRV, we drastically reduced the search space. The resulting application not only solves the puzzles but provides a rich, interactive, and measurable comparison of core AI paradigms.